DETALLES TECNICOS PROVETCARE

Codigo, APIs, Funciones, Frameworks y Middlewares

1. FRONTEND - 12 PAGINAS

Framework: React 18 con Vite

Cada pagina es un componente funcional de React que usa hooks para manejar estado y efectos.

Pagina	Archivo	Hooks Usados	APIs Llamadas
Login	Login.jsx	useState, useNavigate, useAuth	POST /api/auth/login POST /api/auth/register
Dashboard	Dashboard.jsx	useState, useEffect, useAuth	GET /api/appointments GET /api/pets
Calendar	Calendar.jsx	useState, useEffect	GET /api/appointments POST /api/appointments
Pets	Pets.jsx	useState, useEffect	GET /api/pets POST /api/pets DELETE /api/pets/:id
Chat	Chat.jsx	useState, useEffect	GET /api/chat/conversations Socket.io
VetDashboard	VetDashboard.jsx	useState, useEffect	GET /api/admin/stats GET /api/appointments
MedicalConsultation	MedicalConsultation.jsx	useState, useEffect	POST /api/medical-records POST /api/prescriptions
MedicalHistory	MedicalHistory.jsx	useState, useEffect, useParams	GET /api/medical-records/:petId
PaymentsPage	PaymentsPage.jsx	useState, useEffect	GET /api/billing POST /api/billing/pay
RegisterVet	RegisterVet.jsx	useState, useNavigate	POST /api/auth/register
LandingPage	LandingPage.jsx	useNavigate	Ninguna (estatica)
NotFound	NotFound.jsx	useNavigate	Ninguna

Contexto Global: AuthContext

`Archivo: client/src/context/AuthContext.jsx`

Proporciona el estado del usuario autenticado a toda la aplicacion. Funciones exportadas:

- login(email, password): Autentica usuario y guarda token
- logout(): Cierra sesion y limpia token
- register(userData): Crea nueva cuenta

2. ENDPOINTS DE API - 11 RUTAS

Framework: Express.js v4.18

Ruta	Archivo	Metodos HTTP	Middleware	Controller
/api/auth	authRoutes.js	POST login POST register GET verify	authenticateToken	authController.js
/api/appointments	appointmentRoutes.js	GET, POST PUT, DELETE	authenticateToken validateAppointment	appointmentController.js
/api/pets	petRoutes.js	GET, POST PUT, DELETE	authenticateToken	petController.js
/api/medical-records	medicalRecordRoutes.js	GET, POST, PUT	authenticateToken requireVet	medicalRecordController.js
/api/prescriptions	prescriptionRoutes.js	GET, POST GET /:id/pdf	authenticateToken requireVet	prescriptionController.js
/api/billing	billingRoutes.js	GET POST pay POST charge	authenticateToken	billingController.js
/api/invoices	invoiceRoutes.js	GET, POST GET /:id/pdf	authenticateToken	invoiceController.js
/api/chat	chatRoutes.js	GET conversations GET messages	authenticateToken	chatController.js
/api/inventory	inventoryRoutes.js	GET medications GET services	authenticateToken	inventoryController.js
/api/admin	adminRoutes.js	GET stats GET users	authenticateToken requireAdmin	adminController.js
/api	ecosystemRoutes.js	GET services	Ninguno	ecosystemController.js

Servidor Principal: server/server.js

```
const express = require('express');
const app = express();
app.use('/api/auth', authRoutes);
app.use('/api/appointments', appointmentRoutes);
```

```
// ... resto de rutas
```

3. SISTEMA DE AUTENTICACION JWT

Libreria: jsonwebtoken v9.0.2

GENERACION DEL TOKEN (authController.js):

```
const jwt = require('jsonwebtoken');
const token = jwt.sign(
{ userId: user.id, email: user.email, role: user.role },
process.env.JWT_SECRET,
{ expiresIn: '7d' }
);
```

VERIFICACION DEL TOKEN (authMiddleware.js):

```
const token = req.headers.authorization?.split(' ')[1];
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = decoded; // { userId, email, role }
```

ENVIO DESDE EL CLIENTE (api.js):

```
axios.interceptors.request.use(config => {
const token = localStorage.getItem('token');
if (token) config.headers.Authorization = `Bearer ${token}`;
return config;
});
```

MIDDLEWARES DE ROLES (permissions.js):

- requireAdmin: Verifica que user.role === 'admin'
- requireVet: Verifica que user.role === 'veterinario' o 'admin'
- requireOwnership: Verifica que el recurso pertenece al usuario

4. CHAT EN TIEMPO REAL

Libreria: Socket.io v4.6.2 (Cliente y Servidor)

SERVIDOR (server.js):

```
const socketIO = require('socket.io');
const io = socketIO(server, { cors: { origin: '*' } });

io.on('connection', (socket) => {
socket.on('join_conversation', (conversationId) => {
socket.join(`conv_${conversationId}`);
});

socket.on('send_message', async (data) => {
// Guardar en BD
io.to(`conv_${data.conversationId}`).emit('new_message', message);
});
});
```

CLIENTE (Chat.jsx):

```
import io from 'socket.io-client';
const socket = io('http://localhost:5000');

socket.emit('join_conversation', conversationId);
socket.on('new_message', (message) => {
setMessages(prev => [...prev, message]);
});
```

Funciones del Chat:

- chatController.getConversations(): Lista todas las conversaciones del usuario
- chatController.getMessages(conversationId): Obtiene historial de mensajes
- chatController.createConversation(): Crea nueva conversacion

5. GENERACION DE DOCUMENTOS PDF

Libreria: PDFKit v0.17.2

SERVICIO (pdfService.js):

```
const PDFDocument = require('pdfkit');
const fs = require('fs');

async function generatePrescriptionPDF(prescriptionData) {
const doc = new PDFDocument();
const filename = `prescription_${Date.now()}.pdf`;
const path = `./uploads/${filename}`;

doc.pipe(fs.createWriteStream(path));
doc.fontSize(20).text('RECETA MEDICA', { align: 'center' });
doc.fontSize(12).text(`Paciente: ${prescriptionData.petName}`);
// ... mas contenido
doc.end();
return path;
}
```

TIPOS DE PDFs GENERADOS:

1. Recetas Medicas (prescriptionController.js):
- Endpoint: GET /api/prescriptions/:id/pdf
- Incluye: Datos del paciente, veterinario, medicamentos, dosis

2. Facturas (invoiceController.js):
- Endpoint: GET /api/invoices/:id/pdf
- Incluye: Items, subtotal, impuestos, total

3. Recibos de Pago:
- Generado automaticamente al procesar pago
- Incluye: Fecha, monto, metodo de pago, firma digital

6. NOTIFICACIONES POR EMAIL

Librerias: Nodemailer v6.9.7 + Node-Cron v3.0.3

CONFIGURACION SMTP (emailService.js):

```
const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransport({
service: 'gmail',
auth: {
user: process.env.EMAIL_USER,
pass: process.env.EMAIL_PASSWORD
}
});
```

ENVIO DE EMAIL (notificationService.js):

```
async function sendAppointmentReminder(appointment) {
const mailOptions = {
from: process.env.EMAIL_USER,
to: appointment.userEmail,
subject: 'Recordatorio de Cita - PROVETCARE',
html: `Tienes una cita manana`
};
await transporter.sendMail(mailOptions);
}
```

CRON JOB (reminderService.js):

```
const cron = require('node-cron');

// Ejecutar diariamente a las 9:00 AM
cron.schedule('0 9 * * *', async () => {
const tomorrow = new Date();
tomorrow.setDate(tomorrow.getDate() + 1);

const appointments = await getAppointments(tomorrow);
appointments.forEach(apt => sendAppointmentReminder(apt));
});
```

7. MEDIDAS DE SEGURIDAD

A. PROTECCION CONTRA SQL INJECTION

Metodo: Consultas parametrizadas con pg

```
// MAL - Vulnerable a SQL Injection:
const query = `SELECT * FROM users WHERE email='${email}'`;

// BIEN - Uso de parametros:
const query = 'SELECT * FROM users WHERE email=$1';
const result = await pool.query(query, [email]);
```

B. HASH DE CONTRASENAS

Libreria: bcryptjs v2.4.3

```
const bcrypt = require('bcryptjs');

// Al registrar:
const hashedPassword = await bcrypt.hash(password, 10);

// Al hacer login:
const isValid = await bcrypt.compare(password, user.password);
```

C. HEADERS DE SEGURIDAD

Libreria: Helmet v7.1.0

```
const helmet = require('helmet');
app.use(helmet());
// Configura automaticamente headers como:
// - X-Content-Type-Options: nosniff
// - X-Frame-Options: DENY
// - Strict-Transport-Security
```

D. RATE LIMITING

Libreria: express-rate-limit v6.11.2

```
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
windowMs: 15 * 60 * 1000, // 15 minutos
max: 100, // limite de 100 peticiones
```

```
message: 'Demasiadas peticiones desde esta IP'
});
app.use(limiter);
```

E. VALIDACION DE DATOS

Libreria: Zod v3.22.4

```
const { z } = require('zod');
const appointmentSchema = z.object({
petId: z.number().positive(),
date: z.string().datetime(),
serviceType: z.enum(['consulta', 'vacuna', 'cirugia'])
});
// En middleware/validators.js:
const validatedData = appointmentSchema.parse(req.body);
```

F. CORS CONFIGURADO

```
const cors = require('cors');
app.use(cors({
origin: 'http://localhost:5173', // Solo frontend permitido
credentials: true
}));
```